

Changelog

Notable code-level fixes and why they happened — separate from `DATA_NOTES.md` (which tracks fixture/eval-case changes). One entry per reported issue: who found it, what was actually wrong, what changed, how it was verified. Newest first.

2026-09-18 — Live run crashed on `deepseek/deepseek-chat-v3.1`: `TypeError` in `_parse_move`

Reported by: a teammate's first live `--all run (BACKEND=live, MODEL=deepseek/deepseek-chat-v3.1, DECISION_MODE=rules)`, which crashed partway through the 50-case set:

```
File "backends.py", line 1223, in _parse_move
    return json.loads(text)
TypeError: the JSON object must be str, bytes or bytearray, not NoneType
```

Not `DECISION_MODE`. The reporter's hunch was that `decision_mode=rules` caused it. Checked and ruled out: `DECISION_MODE` only changes which prompt text `agent.py` sends (`prompt.build_system_prompt`) - it never touches `backends.py`'s response parsing, which is identical in both modes. Traced the actual path instead: `LiveBackend.next_move()` → `_live_call()` → `_parse_move(raw)`. `_live_call()` read `payload["choices"][0]["message"]["content"]` and handed it straight to `_parse_move`, which only caught `json.JSONDecodeError` around `json.loads(text)`. When `OpenRouter` returned `content: null` for a turn - not unusual for some models on certain finish reasons (content filtering, a reasoning-only turn, an empty completion) - `json.loads(None)` raises `TypeError`, not `JSONDecodeError`, so it escaped the except clause uncaught and took the whole run down.

The fix (`backends.py`): - `_parse_move(text)` now checks `isinstance(text, (str, bytes, bytearray))` first. Non-string content is treated the same as unparseable JSON - a gradeable `escalate` record naming the actual type received - instead of crashing the run. - `_live_call()` now checks for `"choices"` in the response before indexing into it, and raises a `RuntimeError` that includes `OpenRouter`'s own `"error"` body if present. A malformed request or provider-side problem (bad model slug, rate limit, no credit) used to surface as a bare `KeyError` pointing at an indexing line with no information about what actually went wrong; now it says so directly. Also switched `content` extraction to `.get("content")` so a response with `message` but no `content` key returns `None` (now handled) instead of a second `KeyError`.

Verified: - `run_eval.py --mode rules` and `--mode model`, full scripted set: **118/118, 100%** both, unaffected (the scripted backend never calls `_live_call/_parse_move`). - `_parse_move(None)` directly: returns a gradeable `escalate` record (`"model returned no usable content (got NoneType instead of text)"`) instead of raising.

2026-09-18 — Added `--auto-approve` to `run_eval.py`

Why now: flagged as an open gap in `docs/MODEL_BATTERY_PLAN.md` and in the "downstream implication" note on the autonomy-gate fix below — once that fix landed, a live run with no approval callback correctly *holds* every `book` case (`gate_held`), which is the right safety behaviour but means nobody could get a meaningful pass rate out of either the D5(b) battery or the rules-vs-model comparison without first wiring something up.

The fix: - `harness.py` — `run_set()` now takes an `approve=None` parameter and passes it straight through to `run_case()` for every trial. Left as `None` (the default), behaviour is unchanged. - `run_eval.py` — new `--auto-approve` flag. When passed, it builds `approve = lambda action, payload: True` and threads it through both the single-case and `set/ --all` run paths. Prints an explicit banner ("SIMULATING approval... not a human review") every time it's active, and records `"auto_approve": true/false` in `results.json` so a marker or teammate reading someone else's results can tell whether a run's booking outcomes reflect simulated approval. - Deliberately **not** a change to `config.py` or `AUTONOMY` — the flag only affects this evaluation harness's own calls into `run_case`, never the production gate itself. On the scripted backend it's a no-op (that backend already auto-approves internally); it only matters on `BACKEND = "live"`.

Verified: - `run_eval.py --auto-approve` (scripted, default `--mode`): **118/118, 100%**, identical to a plain run — confirms no regression on the path that already auto-approved. - `run_eval.py --mode model` and plain `run_eval.py` (no flag), rerun after this change: **118/118, 100%** both — `--mode` and default-mode parsing unaffected by the new flag's argument handling. - `results.json`'s new `auto_approve` field reads `false` on a normal run and `true` under the flag, confirmed by inspection.

Still open: this only removes the *harness*-side blocker. Whoever runs the live battery still needs their own `OPENROUTER_API_KEY` (own shell, never committed) and their model's price added to `config.PRICES` first — see `docs/MODEL_BATTERY_PLAN.md` items 1–2.

2026-09-17 — Live-run cost was unmeasurable; per-model pricing was flat

Why this was checked now: before anyone spends real API budget on the D5(b) live battery, worth confirming the numbers that come back are actually trustworthy - latency and cost specifically.

Latency: already fine. `agent.py` records real wall-clock `seconds` per run (`time.time()` at start and end). For a live call this is genuine API latency. No change needed.

Cost: two separate problems, both real.

1. `LiveBackend.token_estimate()` was still the scaffold's stub, returning `(0, 0)` unconditionally. Every live run would have reported `tokens_in=0, tokens_out=0,`

`cost_usd=$0.00` regardless of actual spend - the scripted backend's fake estimator was harmless (its numbers were never claimed as measured), but this one would have silently looked like a real, trustworthy zero.

2. `config.PRICE_IN / PRICE_OUT` was one flat rate, always the cheap tier's price, regardless of `config.MODEL`. Six team members are each about to set `MODEL` to something different for their battery slot - this would have silently priced 5 of 6 people's runs at the wrong tier's rate.

The fix: - `backends.py` — `_live_call()` now returns the OpenRouter response's own `usage` field (`prompt_tokens` , `completion_tokens`) alongside the content, instead of discarding it. `LiveBackend` stores the most recent call's usage and `token_estimate()` returns it - measured, not estimated, from the API's own accounting. - `config.py` — `PRICE_IN / PRICE_OUT` replaced with a `PRICES` dict keyed by model name, and a `price_for(model)` lookup that **fails loudly** (`SystemExit` , not a silent wrong number) if a model's price hasn't been entered yet. Whoever sets `MODEL` for their battery slot must add its price pair to `PRICES` first, or the run refuses to proceed. - `agent.py` — cost is now computed from `config.price_for(config.MODEL)` instead of the flat rate.

Verified: - Scripted set, both modes, unaffected: **118/118, 100%** (unchanged - `config.MODEL` 's default stays priced, so nothing about the existing scripted runs changed). - `config.price_for()` on an unlisted model raises immediately with instructions, rather than silently defaulting. - Mocked a full live run (REF-5620 , five turns of realistic OpenRouter `usage` numbers, no real key/network needed to prove the wiring): `tokens_in / tokens_out` matched the sum of the mocked usage exactly, and `cost_usd` matched the hand-computed expected cost exactly.

Still needed before the live battery actually runs: each member adds their model's real price pair (input and output, checked against OpenRouter's own pricing page) to `config.PRICES` before running their slot.

2026-09-17 — Three teammate-added cases (REF-6023 , REF-6027 , REF-6030) mislabelled by a synonym gap

Found by: the independent `resolve_routing` cross-check this repo runs against every label (not just `check_my_data.py` 's existence check) — see `docs/TEST_CASE_MAP.md` / `docs/DATA_NOTES.md` for the method.

The bug: all three referrals described a cardiology complaint as "*shortness of breath*" or "*dyspnea*" — clinically identical to "*breathlessness*" , but not the literal substring `check_referral_criteria` matches against CARD's `treats` list. So `right_department` came back `False` , and each case actually resolved to `escalate / specialty_mismatch` **before** it ever reached the check the case was written to test:

- REF-6023 — labelled `request_information /BNP-01`; actually resolved to `escalate / specialty_mismatch` .

- REF-6027 — same root cause, same actual resolution.
- REF-6030 — labelled book ; actually resolved to escalate / specialty_mismatch . This one visibly failed the code check (117/118) because its script tries book_slot , which the route-consistency guardrail correctly blocked — the guardrail doing exactly its job, not a separate bug.

A second, independent bug found on REF-6030 while fixing the first: its script called lookup_patient with patient_id: "P-1233" , but the referral's actual patient_id is P-1215 . The scripted backend doesn't cross-check that a call's arguments match the referral it's for, so this ran without erroring — it just quietly looked up the wrong patient. Didn't change this case's outcome (neither patient has a conflicting appointment), but the decision record would have cited the wrong patient's duplicate check.

The fix: - make_fixtures_B.py — replaced "shortness of breath"/"dyspnea" with "breathlessness" in all three clinical_summary fields. No other change to the referrals (specialty, tests, patient, dates all untouched). - backends.py (REF-6030) — corrected lookup_patient 's patient_id to P-1215 , and updated its reason / thought text: the actual matching urgency trigger is "rapidly worsening" , not "worsening over days" (both are valid urgent-band triggers in the text, but only one is actually present — the label had cited the wrong one). - expected_outcomes_B.json — same trigger-wording correction in REF-6030 's must_record .

Verified: - check_my_data.py → "Your data hangs together." - Independent resolve_routing cross-check, all 50 labelled cases: **0 mismatches** (was 3). - run_eval.py --mode rules and --mode model , full set: **118/118 trials, 100%**, both modes (was 117/118).

2026-09-17 — Autonomy gate silently auto-approved on the live backend

Reported by: the guardrails owner, reviewing agent.py while building the D3(b) guardrail checklist (13 tests passing at the time of report).

The report: agent.py 's comment said the auto-approval fallback was "for the scripted backend," but the code applied it unconditionally:

```
if approve is None:
    approve = lambda action, payload: True
```

Since harness.py / run_eval.py never pass an approve callback at all, every run - scripted or live - hit this fallback. In AUTONOMY="confirm" mode, that meant a live run with no approval wiring would silently auto-book instead of holding, defeating the whole point of the confirm gate.

Verified before changing anything: - guardrails.py 's gate() already fails closed on its own - bool(approve and approve(...)) is False when approve is None . No change needed there. - The *only* thing causing the bypass was agent.py 's override turning None into an

always- `True` lambda before it ever reached `gate()` . - `backend = make_backend(...)` is already constructed earlier in `run_case()` , so `backend.name` was available to condition on without restructuring anything.

The fix (`agent.py` , `run_case()`): the fallback now only fires when `backend.name == "scripted"` :

```
if approve is None and backend.name == "scripted":
    approve = lambda action, payload: True
```

On the live backend, a missing `approve` now stays `None` , and `gate()` 's existing logic holds the booking correctly.

Regression check — scripted backend, both decision modes, full 50-case set: **117/118 trials, unchanged** from before the fix (the one failing trial, `REF-6030` , is a separate known issue - see the repo's recent commit history, unrelated to this change).

New-behaviour check — simulated a live backend replaying a real, correct move sequence (`REF-5620` , which genuinely resolves to "book") with no `approve` callback supplied:

```
decision: escalate
stopped_by: gate_held
guardrails_fired: [{'guardrail': 'gate_held', 'detail': 'book_slot (autonomy=c
```

Confirms the booking now correctly holds instead of silently going through.

Downstream implication, not yet acted on: once the live battery (D5b) actually runs, whoever calls `run_case / run_set` against `BACKEND="live"` needs to explicitly supply an `approve` callback (e.g. `lambda a, p: True` , mirroring what the scripted path did implicitly) if the goal is measuring *decision quality* rather than exercising the real approval flow. Without it, every live "book" case will now correctly show as held/escalated - which is the right safety behaviour, but will read as a lower pass rate if nobody accounts for it. Worth deciding before the live battery is run, not after.

Next step (owned by the reporter): add a guardrail-checklist case for the missing-approval-holds behaviour and rerun the D3 checklist.